

---

# **LAPORAN PRAKTIKUM INTERNET OF THINGS**

---



**Disusun oleh :**

**Nama : Restu anggara**

**NPM : 25782051**

**Kelas : TRI-3B**

**Dosen :**

**Dr.Ir.Septafiansyah Dwi Putra,S.T.,M.T.**

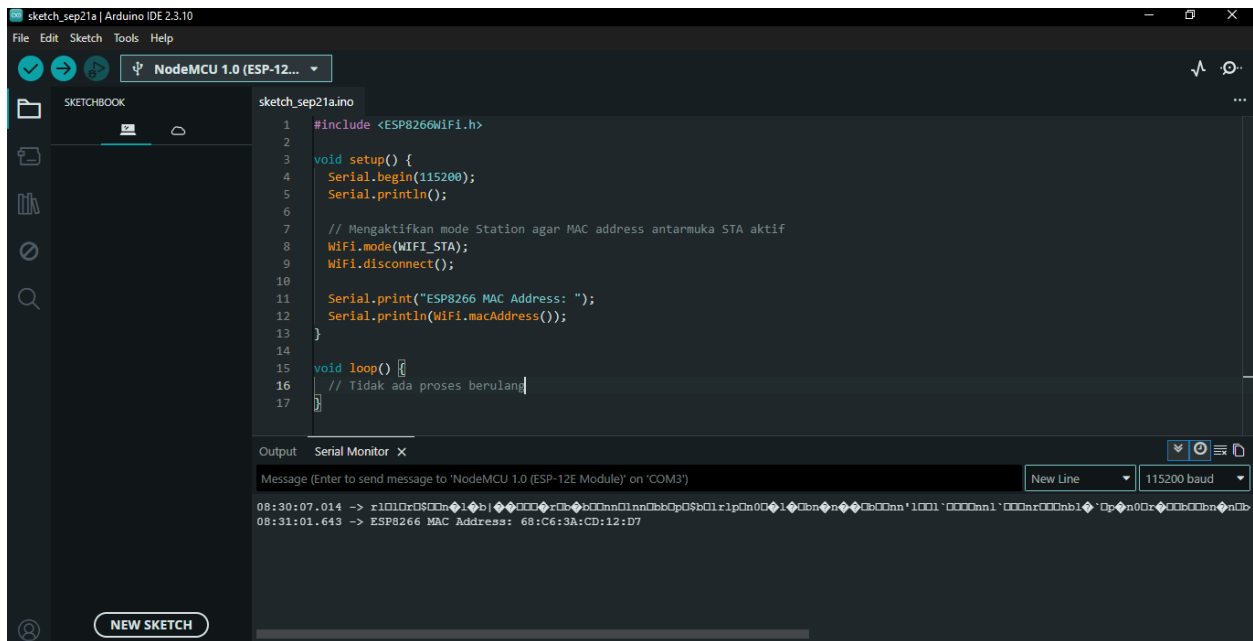
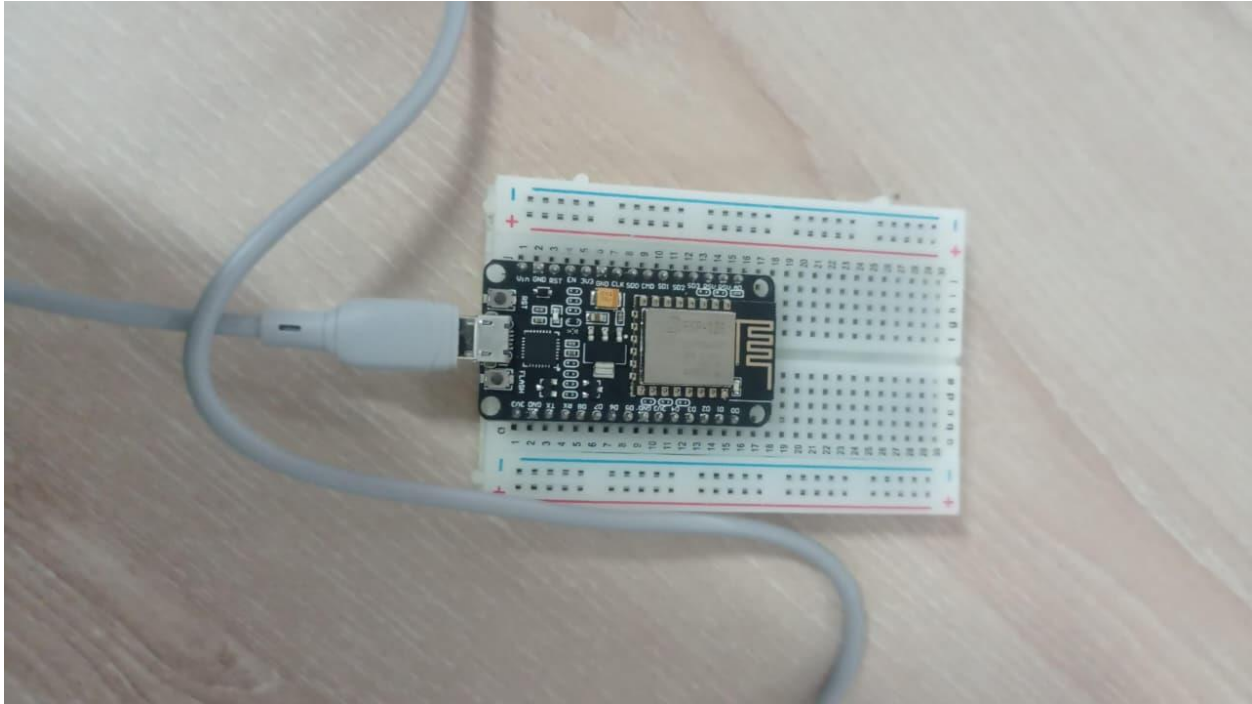
**Agus Ambarwari, S.Pd.,M.Kom.**

**Andi Saptono,S.Kom**

**TEKNOLOGI REKAYASA INTERNET  
JURUSAN TEKNOLOGI INFORMASI  
POLITEKNIK NEGERI LAMPUNG**

**2026/2027**

### 5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU



### Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program pembaca MAC Address berikut:
3. Upload program ke masing-masing board NodeMCU dalam kelompok Anda secara bergantian.
4. Buka Serial Monitor pada kecepatan 115200 baud, lalu tekan tombol fisik RST pada board.

5. Catat 6 pasang digit heksadesimal yang muncul di layar mac yang saya dapatkan **68:C6:3A:CD:12:D7**
6. Beri label fisik pada board (misal: "Node 1", "Node 2", "Hub Pusat") beserta alamat MAC masing-masing agar tidak tertukar saat perakitan.

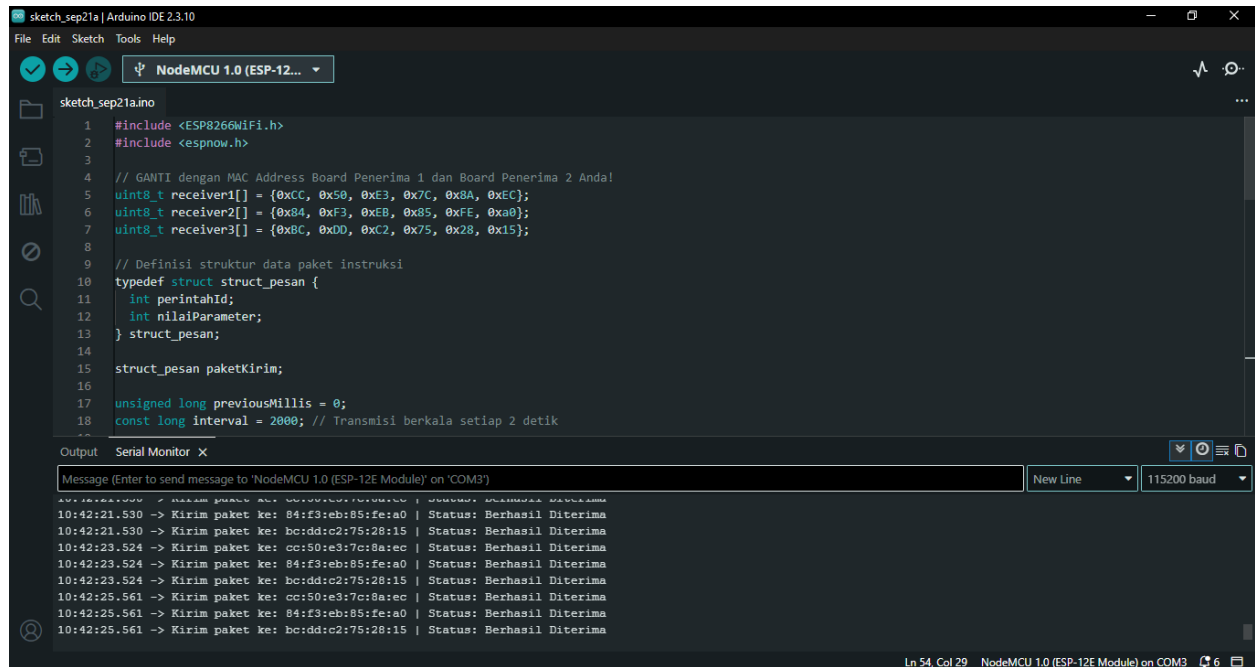
#### **Penjelasan Singkat Kode:**

- `WiFi.mode(WIFI_STA);` : Menyiapkan modul radio Wi-Fi internal ke dalam mode *Station* agar alamat perangkat keras (*Hardware MAC*) siap dialokasikan oleh SDK.
- `WiFi.disconnect();` : Memastikan mikrokontroler tidak mencoba melakukan proses penyambungan ke jaringan luar.
- `WiFi.macAddress();` : Mengambil string alamat fisik 48-bit bawaan pabrik yang bersifat unik secara global.

#### **Kesimpulan:**

setiap board NodeMCU memiliki **MAC Address yang unik** sebagai identitas perangkat dalam jaringan. Pada praktikum ini, MAC Address yang diperoleh adalah **68:C6:3A:CD:12:D7**. MAC Address tersebut dapat digunakan untuk mengenali dan membedakan setiap NodeMCU saat digunakan dalam komunikasi jaringan, sehingga proses perakitan dan konfigurasi antar-node dapat dilakukan dengan lebih mudah dan tidak tertukar.

## 5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)



The screenshot shows the Arduino IDE interface. The sketch file is 'sketch\_sep21a.ino'. The code includes `<ESP8266WiFi.h>` and `<espnow.h>`. It defines three MAC addresses for receivers: `receiver1` (0xCC, 0x50, 0xE3, 0x7C, 0x8A, 0xEC), `receiver2` (0x84, 0xF3, 0xEB, 0x85, 0xFE, 0xA0), and `receiver3` (0x8C, 0x00, 0xC2, 0x75, 0x28, 0x15). A struct `struct_pesan` is defined with `int perintahId` and `int nilaiParameter`. A `struct_pesan paketKirim` is declared. The code also sets `previousMillis = 0` and `interval = 2000` for periodic transmission every 2 seconds.

The Serial Monitor shows the following output:

```
10:42:21.530 -> Kirim paket ke: 84:f3:eb:85:fe:a0 | Status: Berhasil Diterima
10:42:21.530 -> Kirim paket ke: bc:dd:c2:75:28:15 | Status: Berhasil Diterima
10:42:23.524 -> Kirim paket ke: cc:50:e3:7c:8a:ec | Status: Berhasil Diterima
10:42:23.524 -> Kirim paket ke: 84:f3:eb:85:fe:a0 | Status: Berhasil Diterima
10:42:23.524 -> Kirim paket ke: bc:dd:c2:75:28:15 | Status: Berhasil Diterima
10:42:25.561 -> Kirim paket ke: cc:50:e3:7c:8a:ec | Status: Berhasil Diterima
10:42:25.561 -> Kirim paket ke: 84:f3:eb:85:fe:a0 | Status: Berhasil Diterima
10:42:25.561 -> Kirim paket ke: bc:dd:c2:75:28:15 | Status: Berhasil Diterima
```

### Langkah Kerja Pemrograman Sisi Pengirim (Controller):

1. Masukkan alamat MAC milik Board Penerima 1 dan Board Penerima 2 ke dalam variabel `receiver1` dan `receiver2`.
2. Format tanda titik dua (:) diubah menjadi bilangan heksadesimal berawalan 0x yang dipisahkan oleh tanda koma.

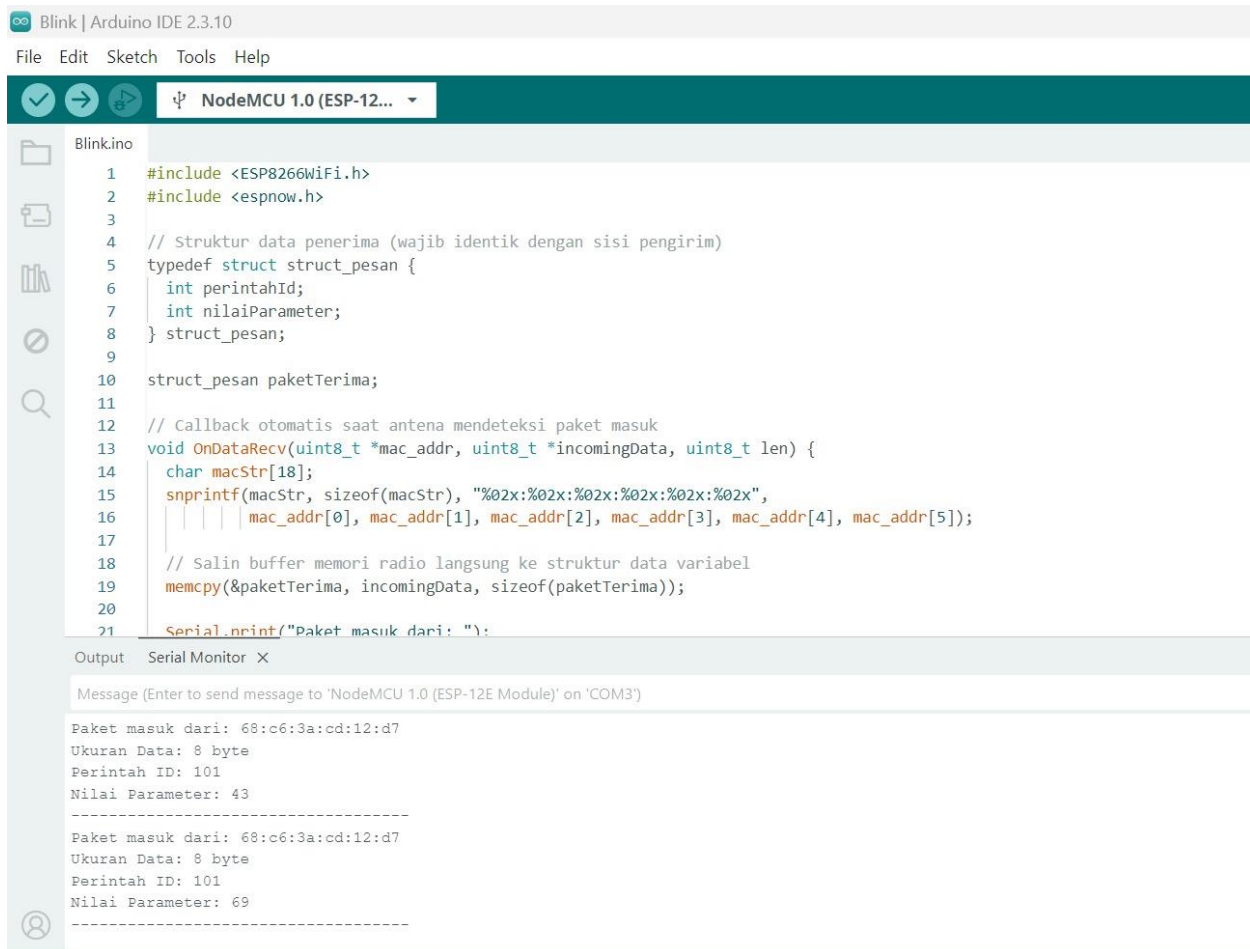
Contoh mac address nya:

```
uint8_t receiver1[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
uint8_t receiver2[] = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF};
```

## Langkah Kerja Pemrograman Sisi Penerima (Slave):

1. Unggah kode sketsa berikut ke **NodeMCU 2** dan **NodeMCU 3**.
2. Board penerima tidak perlu mengetahui alamat MAC milik Controller.

### Node mcu 2



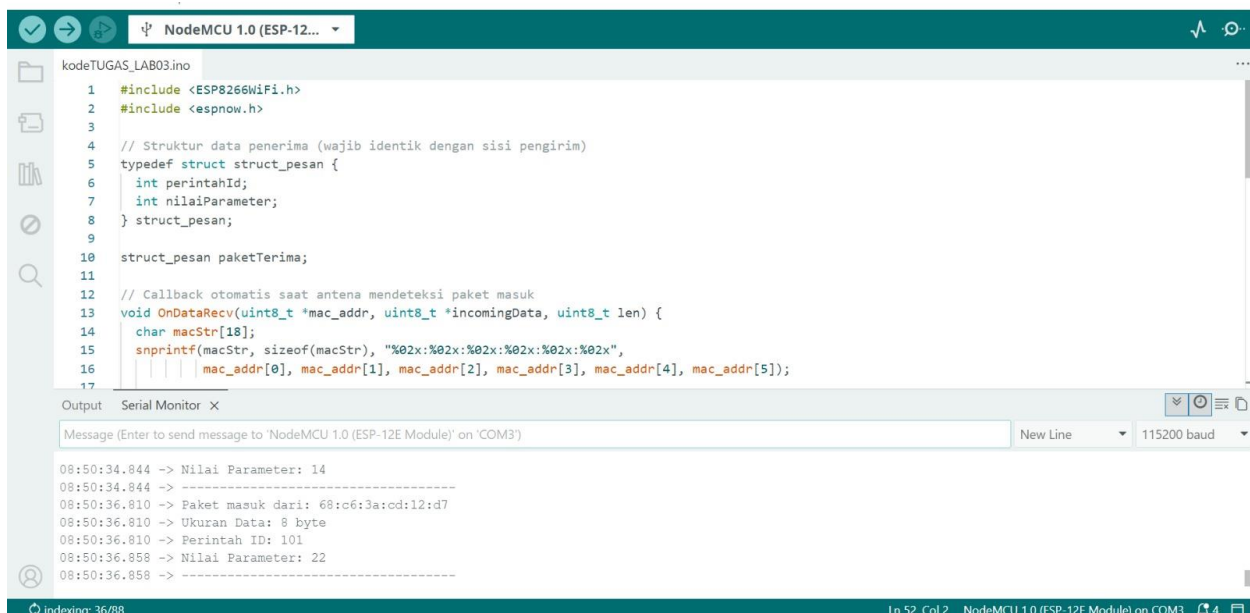
```
Blink.ino
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // Struktur data penerima (wajib identik dengan sisi pengirim)
5  typedef struct struct_pesan {
6      int perintahId;
7      int nilaiParameter;
8  } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18     // Salin buffer memori radio langsung ke struktur data variabel
19     memcpy(&paketTerima, incomingData, sizeof(paketTerima));
20
21     Serial.print("Paket masuk dari: ");
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Paket masuk dari: 68:c6:3a:cd:12:d7  
Ukuran Data: 8 byte  
Perintah ID: 101  
Nilai Parameter: 43  
-----  
Paket masuk dari: 68:c6:3a:cd:12:d7  
Ukuran Data: 8 byte  
Perintah ID: 101  
Nilai Parameter: 69  
-----

### NodeMCU 3



```
kodeTUGAS_LAB03.ino
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // Struktur data penerima (wajib identik dengan sisi pengirim)
5  typedef struct struct_pesan {
6      int perintahId;
7      int nilaiParameter;
8  } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18     // Salin buffer memori radio langsung ke struktur data variabel
19     memcpy(&paketTerima, incomingData, sizeof(paketTerima));
20
21     Serial.print("Paket masuk dari: ");
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

08:50:34.844 -> Nilai Parameter: 14  
08:50:34.844 -> -----  
08:50:36.810 -> Paket masuk dari: 68:c6:3a:cd:12:d7  
08:50:36.810 -> Ukuran Data: 8 byte  
08:50:36.810 -> Perintah ID: 101  
08:50:36.858 -> Nilai Parameter: 22  
08:50:36.858 -> -----

## Nodemcu 4

The image shows the Arduino IDE interface with a C++ sketch for an ESP-12E module. The sketch is named "Blink.ino" and is located in the "NodeMCU 1.0 (ESP-12E Module)" directory. The code includes a header file "Blink.h" and a "void setup()" function that initializes the serial port and sets the role to "ESP\_NOW\_SLAVE". The "void loop()" function contains a comment indicating that the loop is empty, as data reception is handled by an interrupt callback.

```
// Blink.ino
// Blinking an LED using the ESP-12E module

#include "Blink.h"

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
}

void loop() {
  // Loop kosong: penerimaan data sepenuhnya ditangani oleh interrupt callback
}
```

The Serial Monitor window shows the output of the code, displaying the MAC address, the role, and the data received from the ESP-12E module. The output is as follows:

```
ESP8266 MAC Address: BC:DD:C2:75:28:15
Paket masuk dari: 68:c6:3a:cd:12:d7
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 10

-----
Paket masuk dari: 68:c6:3a:cd:12:d7
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 71
-----
```

### Penjelasan Fungsi Khusus ESP-NOW:

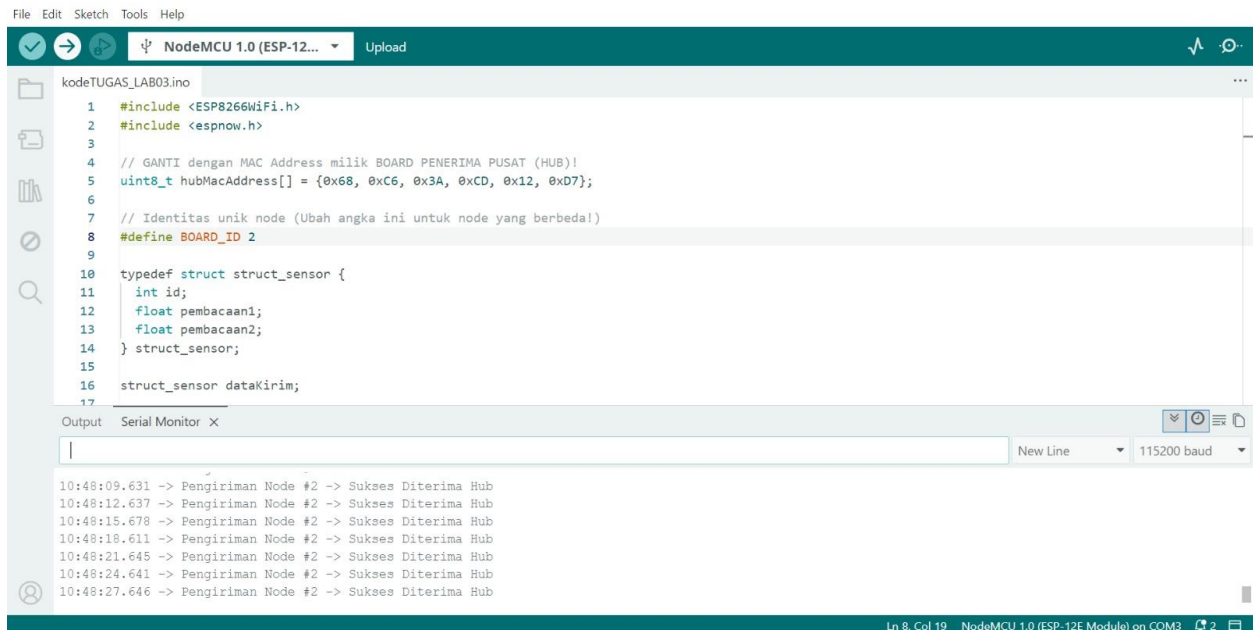
- `esp_now_init()` : Menginisialisasi *firmware stack* ESP-NOW pada chip ESP8266. Mengembalikan nilai 0 jika berhasil.
- `esp_now_set_self_role()` : Mengonfigurasi peran operasional NodeMCU (`ESP_NOW_ROLE_CONTROLLER` untuk pengirim, `ESP_NOW_ROLE_SLAVE` untuk penerima).
- `esp_now_register_send_cb(OnDataSent)` : Mendaftarkan fungsi *callback* yang akan dipanggil sesaat setelah gelombang radio memancarkan paket untuk memastikan laporan ACK diterima atau gagal.
- `esp_now_register_rcv_cb(OnDataRcv)` : Mendaftarkan fungsi *callback* interupsi saat paket radio masuk ke antena penerima.
- `esp_now_add_peer(mac, role, channel, key, key_len)` : Mendaftarkan perangkat tujuan ke dalam tabel routing memori chip.
- `esp_now_send(0, data, size)` : Mengirimkan muatan data. Memasukkan angka 0 pada argumen pertama berarti paket dikirimkan berurutan ke **seluruh** *peer* yang terdaftar.
- `memcpy(&paketTerima, incomingData, sizeof(paketTerima))` : Menyalin deretan byte mentah dari penyangga memori (*buffer*) ke dalam variabel struktur `paketTerima`.

### Kesimpulan:

**ESP-NOW** memungkinkan beberapa perangkat NodeMCU berkomunikasi secara langsung tanpa menggunakan router atau jaringan Wi-Fi biasa. Fungsi seperti `esp_now_init()`, pengaturan role, callback pengiriman dan penerimaan, serta pendaftaran peer digunakan untuk mengatur proses komunikasi antarperangkat. Dengan `esp_now_send()` data dapat dikirim ke

perangkat yang telah terdaftar, sedangkan memcpy() digunakan untuk mengambil data yang diterima dan menyimpannya ke dalam struktur yang sesuai. Dengan demikian, ESP-NOW cocok digunakan untuk komunikasi data antar-NodeMCU yang cepat dan sederhana.

## 5.5.3 Praktikum 3: Pengumpulan Data Telemetry Many-to-One (ESP-NOW)

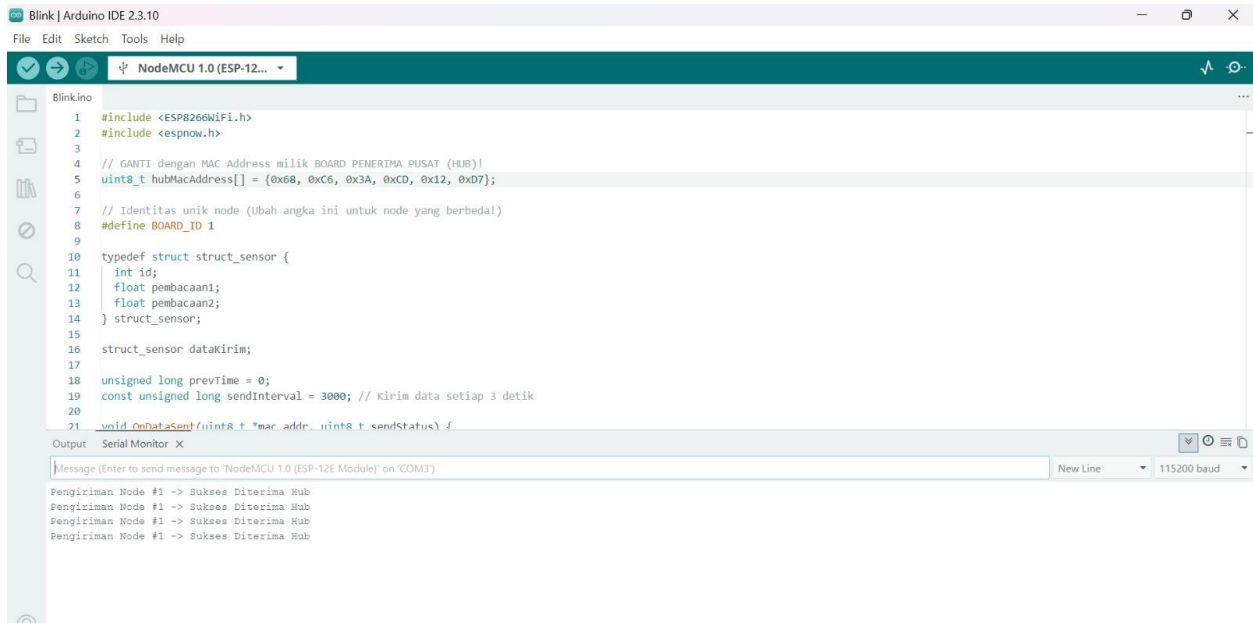


The screenshot shows the Arduino IDE interface with the file `kodeTUGAS_LAB03.ino` open. The code is for an ESP8266 module and includes the following:

```
1 #include <ESP8266WiFi.h>
2 #include <espnw.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
5 uint8_t hubMacAddress[] = {0x68, 0xC6, 0x3A, 0xCD, 0x12, 0xD7};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda!)
8 #define BOARD_ID 2
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
```

The Serial Monitor shows the following output:

```
10:48:09.631 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:12.637 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:15.678 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:18.611 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:21.645 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:24.641 -> Pengiriman Node #2 -> Sukses Diterima Hub
10:48:27.646 -> Pengiriman Node #2 -> Sukses Diterima Hub
```

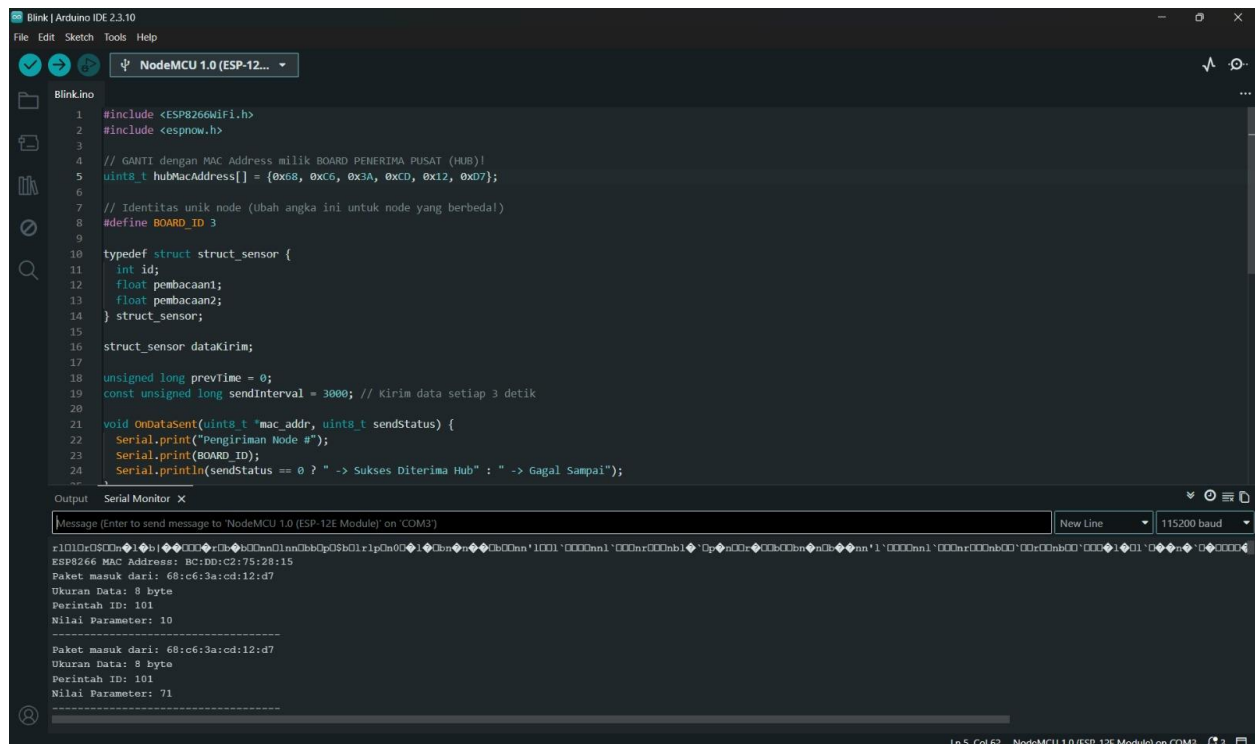


The screenshot shows the Arduino IDE interface with the file `Blink.ino` open. The code is for an ESP8266 module and includes the following:

```
1 #include <ESP8266WiFi.h>
2 #include <espnw.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
5 uint8_t hubMacAddress[] = {0x68, 0xC6, 0x3A, 0xCD, 0x12, 0xD7};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda!)
8 #define BOARD_ID 1
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void onDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
```

The Serial Monitor shows the following output:

```
Message [Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3']
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
```



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
5 uint8_t hubMacAddress[] = {0x68, 0xc6, 0x3a, 0xcd, 0x12, 0xd7};
6
7 // Identitas unik node (ubah angka ini untuk node yang berbeda)
8 #define BOARD_ID 3
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void onDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
22     Serial.print("Pengiriman Node #");
23     Serial.print(BOARD_ID);
24     Serial.println(sendStatus == 0 ? " -> Sukses Diterima Hub" : " -> Gagal Sampai");
25 }
```

Output Serial Monitor

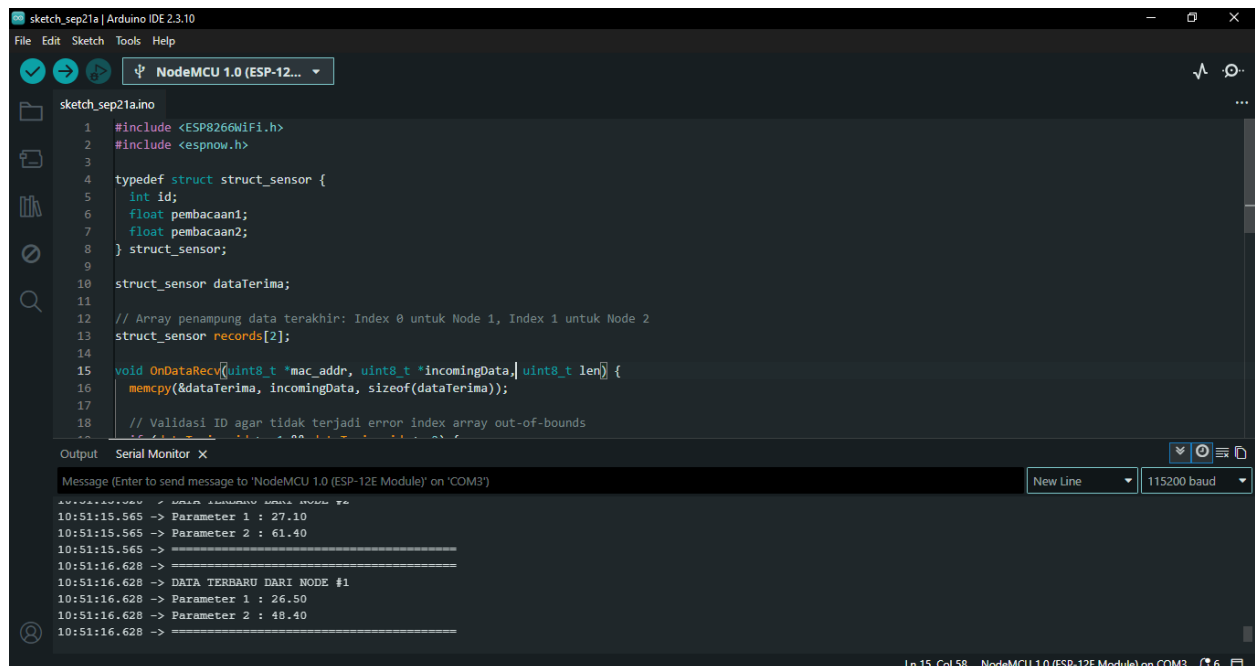
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

ESP8266 MAC Address: BC:DD:C2:75:28:15  
Paket masuk dari: 68:c6:3a:cd:12:d7  
Ukuran Data: 8 byte  
Perintah ID: 101  
Nilai Parameter: 10  
-----  
Paket masuk dari: 68:c6:3a:cd:12:d7  
Ukuran Data: 8 byte  
Perintah ID: 101  
Nilai Parameter: 71  
-----

## Langkah Kerja Pemrograman Sisi Node Pengirim:

1. Masukkan alamat MAC milik **Board Penerima Pusat (Hub)** pada variabel `hubMacAddress`.
2. Pada Board Pengirim 1, atur: `#define BOARD_ID 1`.
3. Pada Board Pengirim 2, atur: `#define BOARD_ID 2`.



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 typedef struct struct_sensor {
5     int id;
6     float pembacaan1;
7     float pembacaan2;
8 } struct_sensor;
9
10 struct_sensor dataTerima;
11
12 // Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
13 struct_sensor records[2];
14
15 void onDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
16     memcpy(&dataTerima, incomingData, sizeof(dataTerima));
17 }
18
19 // Validasi ID agar tidak terjadi error index array out-of-bounds
```

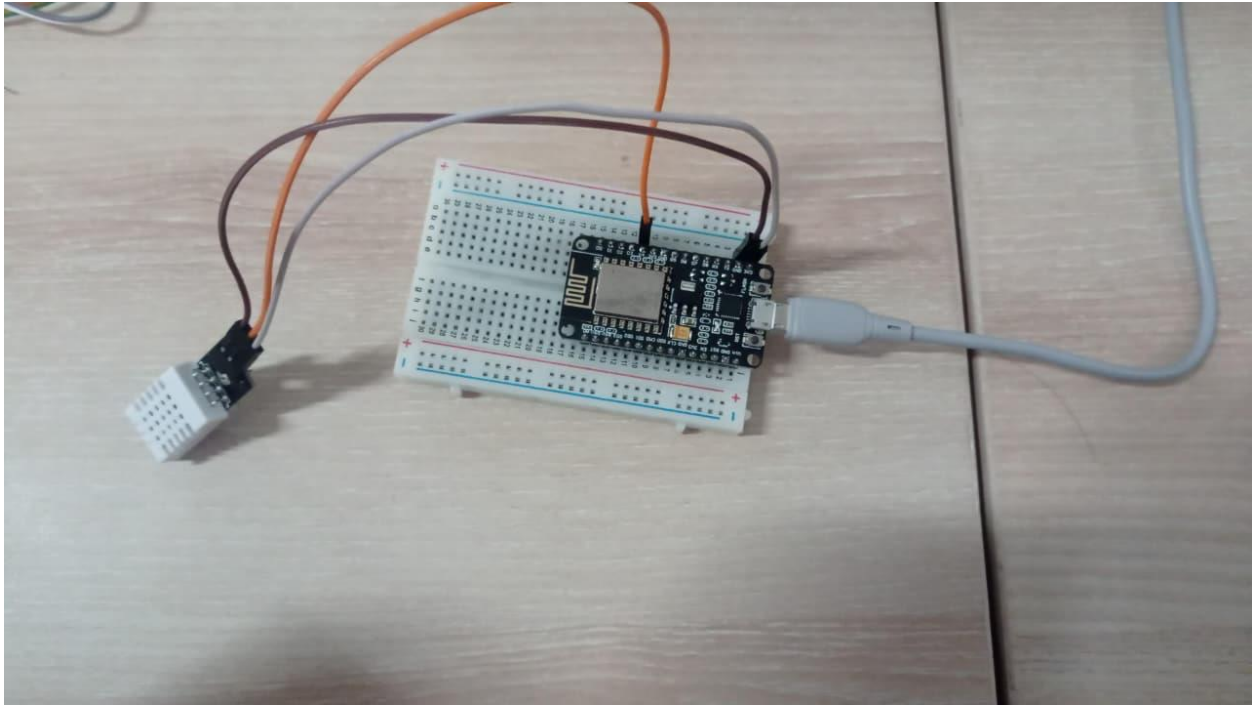
Output Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

10:51:15.565 -> Parameter 1 : 27.10  
10:51:15.565 -> Parameter 2 : 61.40  
10:51:15.565 -> =====  
10:51:16.628 -> =====  
10:51:16.628 -> DATA TERBARU DARI NODE #1  
10:51:16.628 -> Parameter 1 : 26.50  
10:51:16.628 -> Parameter 2 : 40.40  
10:51:16.628 -> =====

### 5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)



```
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12E Module)

sketch_sep2ta.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Lab5RSAR" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "123" // Kata sandi jaringan Mesh
7 #define MESH_PORT 8080 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;

Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3') New Line 115200 baud
10:58:14.988 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.7,"kelembapan":43.7}
10:58:17.971 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.7,"kelembapan":42.5}
10:58:21.014 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.7,"kelembapan":42}
10:58:23.967 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.7,"kelembapan":41.8}
10:58:26.989 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.7,"kelembapan":42.2}
10:58:29.970 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.7,"kelembapan":42.4}
10:58:32.967 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.7,"kelembapan":42.3}
10:58:35.988 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.7,"kelembapan":42.3}
```

#### Instruksi Perangkaian Perangkat Keras:

- Pasang sensor **DHT11 / DHT22** pada masing-masing board NodeMCU:
  - Pin **VCC** sensor ke pin **3V3 (3.3V)** NodeMCU.
  - Pin **GND** sensor ke pin **GND** NodeMCU.
  - Pin **DATA / OUT** sensor ke pin **D4 (GPIO 2)** NodeMCU.

### Langkah Kerja Pemrograman:

1. Unggah sketsa berikut ke **seluruh board** dalam kelompok Anda.
2. Berikan nama pengenalan unik pada variabel nodeName di masing-masing board (misal: "Node-1", "Node-2", "Node-3").
3. Jika menggunakan varian DHT22, ubah #define DHTTYPE DHT11 menjadi DHT22.

### Kesimpulan:

dua NodeMCU sebagai node sensor dapat mengirimkan data telemetry secara langsung ke satu NodeMCU sebagai **Hub Konsentrator** tanpa menggunakan router. Setiap node memiliki BOARD\_ID yang berbeda sehingga data dapat dikenali berdasarkan sumber pengirimnya.

Hub kemudian menerima, memvalidasi, dan menyimpan data terakhir dari masing-masing node menggunakan array records. Dengan sistem ini, beberapa node sensor dapat mengirimkan data ke satu pusat secara terorganisir, sehingga cocok digunakan untuk sistem **monitoring lingkungan dan pengumpulan data sensor secara terpusat**.

## 5.6 Latihan

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:
  - i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
  - ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?  
Ya, benar. Ketika kabel USB dari salah satu board penerima (misalnya NodeMCU 2) dicabut dari sumber daya, status pengiriman untuk MAC address board tersebut pada Serial Monitor NodeMCU Pengirim akan berganti menjadi **"Gagal (Tidak Terjangkau)"**.
  - iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

ESP-NOW dapat mendeteksi kegagalan transmisi **meskipun tidak menggunakan TCP** karena ESP-NOW bekerja pada **Layer 2 (MAC)** dan memiliki mekanisme **ACK (Acknowledgment)** pada komunikasi Wi-Fi.

Ketika sebuah ESP mengirim data ke perangkat tujuan:

1. **ESP pengirim mengirim frame MAC** melalui ESP-NOW.
2. **Perangkat tujuan menerima frame** dan, jika berhasil diterima, akan mengirimkan **ACK** pada tingkat MAC.
3. ESP pengirim menunggu ACK tersebut dalam waktu tertentu.
4. Jika **ACK diterima**, pengiriman dianggap berhasil.
5. Jika **ACK tidak diterima** karena perangkat mati, berada di luar jangkauan, gangguan sinyal, atau terjadi kehilangan frame, ESP-NOW akan melaporkan **kegagalan pengiriman** melalui status callback pengiriman.

Jadi, **TCP tidak diperlukan** untuk mengetahui keberhasilan pengiriman. TCP bekerja di **Layer 4 (Transport)**, sedangkan ESP-NOW menggunakan mekanisme ACK pada **Layer 2 (MAC)** untuk memastikan apakah frame berhasil diterima oleh perangkat tujuan.

- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan `painlessMesh` pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID: ....
  - a. Tekan tombol fisik RST atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh `callback changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?

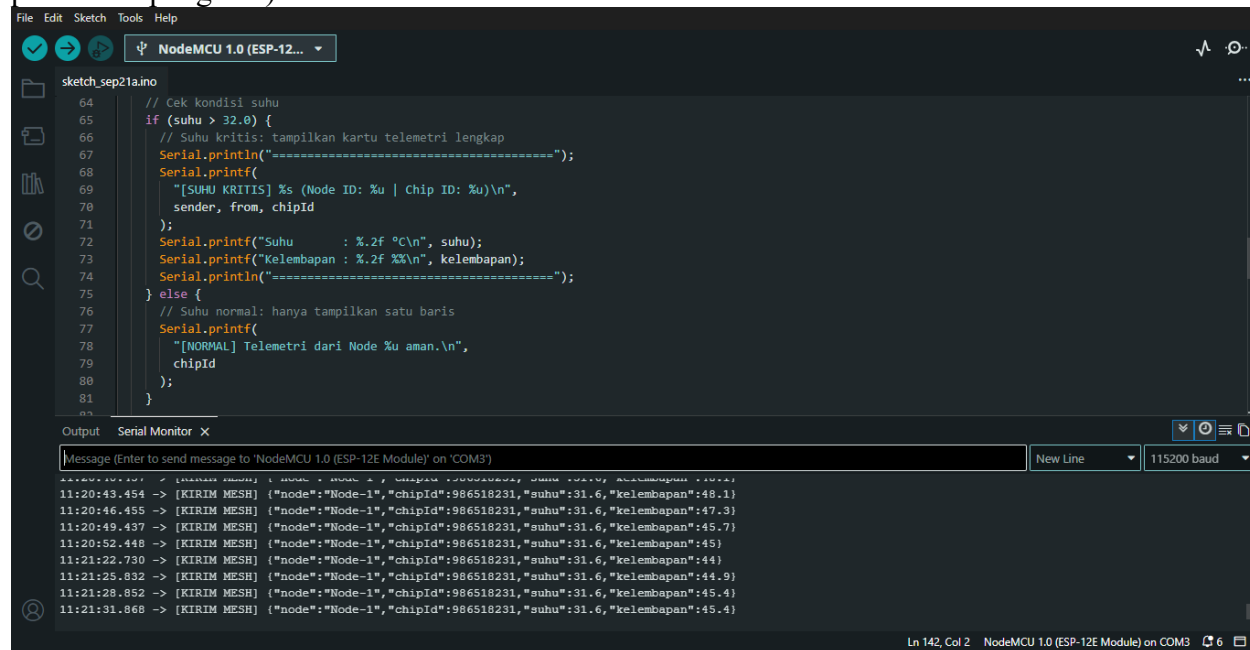
**Waktu yang dibutuhkan sekitar 10 detik.** Setelah salah satu board Node dimatikan atau di-reset, board yang masih menyala akan mendeteksi perubahan koneksi dan memanggil `changedConnectionCallback()` setelah mekanisme timeout koneksi terlewati.

- b. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

**Waktu yang dibutuhkan sekitar 7 detik.** Setelah daya board disambungkan kembali, node melakukan proses pencarian dan koneksi secara otomatis sehingga kembali terdaftar dalam topologi mesh tanpa perlu melakukan konfigurasi ulang jaringan.

• **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.

- i. Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).



```
// Cek kondisi suhu
if (suhu > 32.0) {
  // Suhu kritis: tampilkan kartu telemetri lengkap
  Serial.println("=====");
  Serial.printf(
    "[SUHU KRITIS] %s (Node ID: %u | Chip ID: %u)\n",
    sender, from, chipId
  );
  Serial.printf("Suhu      : %.2f °C\n", suhu);
  Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
  Serial.println("=====");
} else {
  // Suhu normal: hanya tampilkan satu baris
  Serial.printf(
    "[NORMAL] Telemetri dari Node %u aman.\n",
    chipId
  );
}
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

11:20:43.454 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.6,"kelembapan":48.1}

11:20:46.455 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.6,"kelembapan":47.3}

11:20:49.437 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.6,"kelembapan":45.7}

11:20:52.448 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.6,"kelembapan":45}

11:21:22.730 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.6,"kelembapan":44}

11:21:25.832 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.6,"kelembapan":44.9}

11:21:28.852 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.6,"kelembapan":45.4}

11:21:31.868 -> [KIRIM MESH] {"node":"Node-1","chipId":986518231,"suhu":31.6,"kelembapan":45.4}

- ii. Jika suhu bernilai normal ( $\leq 32.0$  °C), program cukup mencetak status singkat satu baris: `[NORMAL] Telemetri dari Node <chipId> aman`.
- iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

Modifikasi dilakukan pada fungsi **receivedCallback()**. Sebelumnya, setiap data yang diterima selalu ditampilkan sebagai telemetry lengkap. Sekarang program terlebih dahulu memeriksa nilai suhu dari node tetangga.

Bagian:

if (suhu > 32.0)

digunakan untuk menentukan apakah suhu berada dalam kondisi kritis.

- Jika **suhu > 32°C**, program menampilkan kartu telemetry lengkap berupa nama node, Node ID, Chip ID, suhu, dan kelembapan.
- Jika **suhu ≤ 32°C**, program hanya menampilkan satu baris:

[NORMAL] Telemetry dari Node <chipId> aman.

Dengan demikian, Node penerima berfungsi sebagai **Pengawas Kondisi Suhu Kritis**. Pengujian dapat dilakukan dengan mendekatkan jari hangat atau meniup sensor DHT22 pada node pengirim hingga suhu yang diterima melebihi 32.0°C.